```
# 🏗️ DIABETIC RETINOPATHY DETECTION - COMPLETE SYSTEM ARCHITECTURE
## Production Implementation (5-Fold Cross-Validation, 25 Epochs Per Fold)

═══════════════════════════════════════════════════════════════════════════════
                              📥 DATA PIPELINE & PREPROCESSING
═══════════════════════════════════════════════════════════════════════════════

┌─────────────────────────────────────────────────────────────────────────────┐
│                           RAW DATA SOURCES                                  │
├─────────────────────────────────────────────────────────────────────────────┤
│                                                                             │
│  Dataset: Benjamin Warner - Resized 2015-2019 Blindness Detection         │
│  Source: Kaggle (EyePACS 2015 + APTOS 2019)                               │
│  Total Images: 38,788 (before filtering)                                  │
│  Final Dataset: 38,625 images (after quality filtering)                   │
│                                                                             │
│  Pre-applied by Dataset:                                                   │
│  • Circle crop (fundus region extraction)                                 │
│  • Resized to max 1024px                                                  │
│  • Ben Graham normalization (ilovescience method)                         │
│                                                                             │
│  Grade Distribution:                                                        │
│  • Grade 0 (No DR):       ~28,000 images (73%)                            │
│  • Grade 1 (Mild):        ~5,200 images (13%)                             │
│  • Grade 2 (Moderate):    ~3,800 images (10%)                             │
│  • Grade 3 (Severe):      ~1,200 images (3%)                              │
│  • Grade 4 (Proliferative): ~425 images (1%)                              │
│                                                                             │
└───────────────────────────────────────┬─────────────────────────────────────┘
                                        │
                                        ▼
┌─────────────────────────────────────────────────────────────────────────────┐
│                    QUALITY FILTERING (preprocess.py)                        │
├─────────────────────────────────────────────────────────────────────────────┤
│                                                                             │
│  validate_image_quality():                                                 │
│  ├─ Too dark: mean brightness < 15                                       │
│  ├─ Low contrast: std deviation < 8                                      │
│  └─ Partial retina: area ratio < 0.3                                     │
│                                                                             │
│  Removed: ~163 images (0.4%)                                               │
│  Final: 38,625 valid images                                                │
│                                                                             │
└───────────────────────────────────────┬─────────────────────────────────────┘
                                        │
                                        ▼
┌─────────────────────────────────────────────────────────────────────────────┐
│              MULTI-RESOLUTION PREPROCESSING (preprocess.py)                 │
```

```
├──────────────────────────────────────────────────────────────────────────────┤
│                                                                              │
│  preprocess_image(img, output_size):                                         │
│  ┌────────────────────────────────────────────────────────────┐              │
│  │ 1. Resize to target (512/768/1024)                         │              │
│  │    • Interpolation: INTER_LANCZOS4                         │              │
│  │                                                            │              │
│  │ 2. Light CLAHE Enhancement                                 │              │
│  │    • Color space: YUV                                      │              │
│  │    • clipLimit: 1.5 (gentle, dataset already enhanced)     │              │
│  │    • tileGridSize: (8, 8)                                  │              │
│  │    • Applied to Y channel only                             │              │
│  └────────────────────────────────────────────────────────────┘              │
│                                                                              │
│  Output: Three resolution sets                                               │
│  ├─ D:/Major_project_shiro_final/preprocessed_512/  (38,625 images)          │
│  ├─ D:/Major_project_shiro_final/preprocessed_768/  (38,625 images)          │
│  └─ D:/Major_project_shiro_final/preprocessed_1024/ (38,625 images)          │
│                                                                              │
└──────────────────────────────────────┬───────────────────────────────────────┘
                                       │
                                       ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│                  CROSS-VALIDATION SPLIT (data_split.py)                      │
├──────────────────────────────────────────────────────────────────────────────┤
│                                                                              │
│  setup_cross_validation(df, n_splits=5):                                     │
│  ├─ Method: StratifiedGroupKFold                                             │
│  ├─ Groups: Patient ID (left/right eye from same patient)                    │
│  ├─ Stratification: Maintains grade distribution per fold                    │
│  └─ Folds: 5 (each ~7,725 images)                                            │
│                                                                              │
│  Per Fold Distribution (~7,725 images):                                      │
│  ├─ Training: ~6,180 images (80%)                                            │
│  └─ Validation: ~1,545 images (20%)                                          │
│                                                                              │
└──────────────────────────────────────┬───────────────────────────────────────┘
                                       │
                                       ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│                    DATA AUGMENTATION (dataset.py)                            │
├──────────────────────────────────────────────────────────────────────────────┤
│                                                                              │
│  DRDataset Class - Fold-Specific Augmentation Strategies:                    │
│                                                                              │
│  ┌────────────────────────────────────────────────────────────┐              │
│  │ FOLD 1 (ConvNeXt):                                         │              │
│  │ • MixUp (alpha=0.4) - blend two images                     │              │
```

```
│  │ • Rotate90 (p=0.5)                              │             │
│  │ • HorizontalFlip (p=0.5)                        │             │
│  │ • VerticalFlip (p=0.5)                          │             │
│  │ • Normalize (ImageNet: μ=[0.485,0.456,0.406])       │             │
│  └─────────────────────────────────────────────────┘             │
│                                                                    │
│  ┌─────────────────────────────────────────────────┐             │
│  │ FOLD 2 (EfficientNet):                          │             │
│  │ • CutMix (alpha=1.0) - cut and paste regions    │             │
│  │ • ShiftScaleRotate (p=0.5)                      │             │
│  │   - shift: ±0.0625, scale: ±0.1, rotate: ±15°       │             │
│  │ • HorizontalFlip (p=0.5)                        │             │
│  │ • VerticalFlip (p=0.5)                          │             │
│  │ • Normalize (ImageNet stats)                    │             │
│  └─────────────────────────────────────────────────┘             │
│                                                                    │
│  ┌─────────────────────────────────────────────────┐             │
│  │ FOLD 3-5: Standard Augmentation (No MixUp/CutMix)   │             │
│  │ • HorizontalFlip (p=0.5)                        │             │
│  │ • VerticalFlip (p=0.5)                          │             │
│  │ • Rotate90 (p=0.3)                              │             │
│  │ • ShiftScaleRotate (p=0.3, gentle)              │             │
│  │ • Normalize (ImageNet stats)                    │             │
│  └─────────────────────────────────────────────────┘             │
│                                                                    │
│  Note: MixUp/CutMix disabled at 1024px stage for stability      │
│                                                                    │
└───────────────────────────────┬──────────────────────────────────┘
                                │
                                ▼
                       ┌───────────────┐
                       │  Batched Tensors │
                       │  (B, 3, H, W)    │
                       │  H,W ∈ {512,     │
                       │   768, 1024}     │
                       └───────────────┘

═══════════════════════════════════════════════════════════════════
                 🧠 NEURAL NETWORK ARCHITECTURE (SOTA_DR_Model)
═══════════════════════════════════════════════════════════════════

                       ┌───────────────┐
                       │  Input Tensor    │
                       │  (B, 3, H, W)    │
                       └───────┬───────┘
                               │
                               ▼
┌──────────────────────────────────────────────────────────────────┐
```

```
│                    BACKBONE NETWORK (backbones.py)                      │
├─────────────────────────────────────────────────────────────────────────┤
│                                                                         │
│  Architecture Selection Per Fold:                                       │
│  ┌──────────────────────┬──────────────────────────────────────────┐    │
│  │ FOLD 1, 3, 5         │ FOLD 2, 4                                │    │
│  ├──────────────────────┼──────────────────────────────────────────┤    │
│  │ ConvNeXtV2-Large     │ EfficientNetV2-L                         │    │
│  │ (fcmae_ft_in1k)      │ (tf_efficientnetv2_l_in21k)              │    │
│  ├──────────────────────┼──────────────────────────────────────────┤    │
│  │ Layers: ~200         │ Layers: 1000+                            │    │
│  │ Channels: 1536       │ Channels: 1280                           │    │
│  │ Params: 198M         │ Params: 118M                             │    │
│  │ Pre-train: IN-1K     │ Pre-train: IN-21K                        │    │
│  │ Drop Path: 0.3       │ Drop Path: 0.3                           │    │
│  │ Grad Checkpoint: ✓   │ Grad Checkpoint: ✓                       │    │
│  └──────────────────────┴──────────────────────────────────────────┘    │
│                                                                         │
│  Configuration:                                                         │
│  • pretrained=True (ImageNet weights)                                   │
│  • num_classes=0 (remove classification head)                           │
│  • global_pool='' (return feature maps)                                 │
│  • drop_path_rate=0.3 (Stochastic Depth for regularization)             │
│                                                                         │
│  Output: Feature Maps (B, C, H/32, W/32)                                │
│  • C = 1536 (ConvNeXt) or 1280 (EfficientNet)                           │
│  • Spatial resolution reduced by 32x                                    │
│  • Example: 1024×1024 input → 32×32 feature maps                        │
│                                                                         │
└────────────────────────────────────┬────────────────────────────────────┘
                                     │
                                     │ Feature Maps (B, C, H', W')
                                     │
                                     ▼
┌─────────────────────────────────────────────────────────────────────────┐
│          FEATURE REFINEMENT GATE (attention.py - FRG)                   │
├─────────────────────────────────────────────────────────────────────────┤
│                                                                         │
│  Purpose: Learn adaptive weighting based on image complexity            │
│                                                                         │
│  Architecture:                                                          │
│  ┌───────────────────────────────────────────────────────────────┐      │
│  │ Input: (B, C, H', W')                                         │      │
│  │   ↓                                                           │      │
│  │ GlobalAvgPool2d → (B, C, 1, 1)                                │      │
│  │   ↓                                                           │      │
│  │ Conv2d(C → C/16, kernel=1) + ReLU                             │      │
│  │   ↓                                                           │      │
```

```
│  │ Conv2d(C/16 → 1, kernel=1) + Sigmoid                    │          │
│  │   ↓                                                       │          │
│  │ weight ∈ [0, 1]                                         │          │
│  └───────────────────────────────────────────────────────────┘          │
│                                                                           │
│  Interpretation:                                                          │
│  • weight ≈ 1.0: Complex image with many lesions                        │
│  • weight ≈ 0.5: Moderate complexity                                    │
│  • weight ≈ 0.0: Simple/healthy fundus                                  │
│                                                                           │
│  Output: Scalar weight (B, 1, 1, 1)                                     │
│                                                                           │
└───────────────────────────────────────┬───────────────────────────────────┘
                                        │
                                        ▼
┌───────────────────────────────────────────────────────────────────────────┐
│           CBAM ATTENTION MODULE (attention.py - CBAM_Module)              │
├───────────────────────────────────────────────────────────────────────────┤
│                                                                           │
│  Convolutional Block Attention Module (Dual Attention)                   │
│                                                                           │
│  ┌───────────────────────────────────────────────────────────┐          │
│  │ CHANNEL ATTENTION BRANCH:                                 │          │
│  │ ────────────────────────────────────────────────────────  │          │
│  │ Input: (B, C, H', W')                                     │          │
│  │   ↓                                                       │          │
│  │ MaxPool2d(HW→1) ──┐                                     │          │
│  │ AvgPool2d(HW→1) ──┤                                     │          │
│  │                  ↓                                        │          │
│  │         Shared MLP (C → C/16 → C)                       │          │
│  │                  ↓                                        │          │
│  │            Element-wise Add                               │          │
│  │                  ↓                                        │          │
│  │              Sigmoid()                                    │          │
│  │                  ↓                                        │          │
│  │         Channel Attention Map (B, C, 1, 1)              │          │
│  └───────────────────────────────────────────────────────────┘          │
│                    ↓ (Multiply with input)                               │
│  ┌───────────────────────────────────────────────────────────┐          │
│  │ SPATIAL ATTENTION BRANCH:                                 │          │
│  │ ────────────────────────────────────────────────────────  │          │
│  │ Input: Channel-refined features                           │          │
│  │   ↓                                                       │          │
│  │ MaxPool(C→1) ──┐                                        │          │
│  │ AvgPool(C→1) ──┤                                        │          │
│  │                ↓                                          │          │
│  │        Concat along channel                               │          │
│  │                ↓                                          │          │
```

```
│   │     Conv2d(2→1, kernel=7×7) + Sigmoid                  │            │
│   │                 ↓                                       │            │
│   │     Spatial Attention Map (B, 1, H', W')               │            │
│   └─────────────────────────────────────────────────────────┘            │
│                   ↓ (Multiply with input)                                │
│                                                                            │
│  Output: Refined Feature Maps (B, C, H', W')                            │
│  • Enhanced lesion regions                                                │
│  • Suppressed background/artifacts                                        │
│                                                                            │
└───────────────────────────────────┬──────────────────────────────────────┘
                                    │
                                    │ Attention-refined features
                                    │
                                    ▼
                          ┌────────────────┐
                          │ GlobalAvgPool2d │
                          │ (B, C, H', W')  │
                          │       ↓         │
                          │ (B, C)          │
                          └───────┬────────┘
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                  MULTI-TASK PREDICTION HEADS (heads.py)                    │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                            │
│  Input: Pooled features (B, C)                                            │
│                                                                            │
│   ┌───────────────────────────────────────────────────────┐              │
│   │ HEAD 1: ORDINAL REGRESSION (CORN)                     │              │
│   │ ────────────────────────────────────────────────────   │              │
│   │ Purpose: Predict DR severity (0→1→2→3→4 ordering)     │              │
│   │                                                         │              │
│   │ Architecture:                                           │              │
│   │ • Linear(C → 512) + LayerNorm + ReLU + Dropout(0.3)   │              │
│   │ • Linear(512 → 256) + LayerNorm + ReLU + Dropout(0.3) │              │
│   │ • Linear(256 → 4)                                      │              │
│   │                                                         │              │
│   │ Output: 4 binary logits (B, 4)                         │              │
│   │ • logit[0]: P(grade > 0) - at least mild DR            │              │
│   │ • logit[1]: P(grade > 1) - at least moderate           │              │
│   │ • logit[2]: P(grade > 2) - at least severe             │              │
│   │ • logit[3]: P(grade > 3) - proliferative DR            │              │
│   │                                                         │              │
│   │ Decoding to class probabilities:                       │              │
│   │ • P(grade=0) = 1 - σ(logit[0])                         │              │
│   │ • P(grade=1) = σ(logit[0]) - σ(logit[1])               │              │
```

```
│   │ • P(grade=2) = σ(logit[1]) - σ(logit[2])                   │           │
│   │ • P(grade=3) = σ(logit[2]) - σ(logit[3])                   │           │
│   │ • P(grade=4) = σ(logit[3])                                 │           │
│   │ (σ = sigmoid function)                                      │          │
│   └─────────────────────────────────────────────────────────────┘          │
│                                                                              │
│   ┌─────────────────────────────────────────────────────────────┐          │
│   │ HEAD 2: BINARY CLASSIFICATION (FOCAL LOSS)                  │          │
│   │ ────────────────────────────────────────────────────────────  │          │
│   │ Purpose: Detect referable DR (grade ≥ 2)                    │          │
│   │                                                               │          │
│   │ Architecture:                                                 │          │
│   │ • Linear(C → 256) + ReLU + Dropout(0.3)                     │          │
│   │ • Linear(256 → 1)                                             │          │
│   │                                                               │          │
│   │ Output: Single logit (B, 1)                                 │          │
│   │ • σ(logit) > 0.5 → Referable DR (needs treatment)         │          │
│   │ • σ(logit) ≤ 0.5 → Non-referable (monitoring only)        │          │
│   └─────────────────────────────────────────────────────────────┘          │
│                                                                              │
│   ┌─────────────────────────────────────────────────────────────┐          │
│   │ HEAD 3: REGRESSION (CONTINUOUS SEVERITY)                    │          │
│   │ ────────────────────────────────────────────────────────────  │          │
│   │ Purpose: Predict continuous severity score [0, 4]           │          │
│   │                                                               │          │
│   │ Architecture:                                                 │          │
│   │ • Linear(C → 256) + ReLU + Dropout(0.2)                     │          │
│   │ • Linear(256 → 1) + Sigmoid × 4.0                           │          │
│   │                                                               │          │
│   │ Output: Continuous score (B, 1) ∈ [0, 4]                   │          │
│   │ • Provides soft severity estimate                           │          │
│   │ • Used for consistency regularization                       │          │
│   └─────────────────────────────────────────────────────────────┘          │
│                                                                              │
└────────────────────────────────────────────────────────────────────────────┘

══════════════════════════════════════════════════════════════════════════════
                          🎯 LOSS FUNCTIONS (losses.py)
══════════════════════════════════════════════════════════════════════════════

┌────────────────────────────────────────────────────────────────────────────┐
│                        MULTI-TASK LOSS COMPOSITION                           │
├────────────────────────────────────────────────────────────────────────────┤
│                                                                              │
│  Total Loss = α·L_CORN + β·L_focal + γ·L_huber + λ_qwk·L_QWK + λ_cons·L_cons│
│                                                                              │
│  Where:                                                                      │
│  • α, β, γ = 1.0 (fixed weights)                                           │
```

```
│  • λ_qwk = {0.0 (epochs 1-10), 0.05 (epochs 11-25)}                         │
│  • λ_cons = {0.01 (epochs 1-10), 0.03 (epochs 11-25)}                       │
│                                                                              │
└──────────────────────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────────────────────┐
│                      L_CORN: CORN ORDINAL LOSS                               │
├──────────────────────────────────────────────────────────────────────────────┤
│                                                                              │
│  Conditional Ordinal Regression Neural Network Loss                          │
│                                                                              │
│  For each threshold k ∈ {1, 2, 3, 4}:                                        │
│    y_binary[k-1] = 1{y_true ≥ k}                                             │
│                                                                              │
│  L_CORN = (1/4) Σ_{k=1}^4 BCE(logit_k, y_binary[k-1])                        │
│                                                                              │
│  With positive class weighting:                                              │
│    pos_weight = 1.5 (current optimal value)                                  │
│                                                                              │
│  Label smoothing applied:                                                    │
│    y_smooth = y * (1 - ε) + 0.5 * ε, where ε = 0.15                          │
│                                                                              │
│  Properties:                                                                 │
│  • Respects ordinal nature (1 < 2 < 3 < 4)                                   │
│  • Each threshold is independent binary classifier                           │
│  • pos_weight handles severe class imbalance (73% Grade 0)                   │
│                                                                              │
└──────────────────────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────────────────────┐
│                   L_focal: FOCAL LOSS FOR BINARY HEAD                        │
├──────────────────────────────────────────────────────────────────────────────┤
│                                                                              │
│  Focal Loss for referable DR detection (grade ≥ 2)                           │
│                                                                              │
│  L_focal = -α_t (1 - p_t)^γ log(p_t)                                         │
│                                                                              │
│  Where:                                                                      │
│    p_t = { p      if y = 1 (referable)                                       │
│          { 1 - p  if y = 0 (non-referable)                                   │
│                                                                              │
│    α_t = { α      if y = 1                                                   │
│          { 1 - α  if y = 0                                                   │
│                                                                              │
│  Hyperparameters:                                                            │
│    α = 0.28  (weight for positive class)                                     │
│    γ = 2.0   (focusing parameter)                                            │
│                                                                              │
```

```
│  Properties:                                                              │
│  • Down-weights easy examples (confident predictions)                     │
│  • Focuses on hard-to-classify borderline cases                          │
│  • α handles class imbalance (only ~14% referable)                       │
│  • γ=2 provides strong focus on hard examples                            │
│                                                                           │
└──────────────────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────────────────┐
│                 L_huber: HUBER LOSS FOR REGRESSION HEAD                   │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                           │
│  Smooth L1 loss for continuous severity prediction                        │
│                                                                           │
│  L_huber(y_pred, y_true) = {                                             │
│    0.5 * (y_pred - y_true)²           if |y_pred - y_true| ≤ δ          │
│    δ * (|y_pred - y_true| - 0.5*δ)    otherwise                         │
│  }                                                                        │
│                                                                           │
│  Where δ = 1.0 (standard setting)                                        │
│                                                                           │
│  Properties:                                                              │
│  • Quadratic for small errors (smooth gradient)                          │
│  • Linear for large errors (robust to outliers)                          │
│  • Less sensitive to extreme predictions than MSE                        │
│                                                                           │
└──────────────────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────────────────┐
│                   L_QWK: DIFFERENTIABLE QWK LOSS                          │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                           │
│  Soft Quadratic Weighted Kappa (approximation for backprop)              │
│                                                                           │
│  1. Convert CORN logits to class probabilities:                          │
│     p_0 = 1 - σ(logit_0)                                                │
│     p_k = σ(logit_{k-1}) - σ(logit_k) for k ∈ {1,2,3}                  │
│     p_4 = σ(logit_3)                                                    │
│                                                                           │
│  2. Build soft confusion matrix:                                         │
│     C[i,j] = Σ_n p_n[j] · 1{y_n = i}                                    │
│                                                                           │
│  3. Compute expected and observed agreement:                             │
│     O = Σ_{i,j} w[i,j] · C[i,j] / N                                     │
│     E = Σ_{i,j} w[i,j] · (Σ_k C[i,k])(Σ_k C[k,j]) / N²                │
│                                                                           │
│  4. Calculate QWK:                                                        │
│     κ = 1 - O/E                                                          │
```

```
│                                                                      │
│  5. Convert to loss:                                                 │
│     L_QWK = 1 - κ                                                   │
│                                                                      │
│  Weight matrix w[i,j] (quadratic penalties):                        │
│     w[i,j] = ((i-j)/(K-1))²  where K=5 classes                     │
│                                                                      │
│  Properties:                                                         │
│  • Directly optimizes evaluation metric                              │
│  • Penalizes large errors more (quadratic weights)                  │
│  • Differentiable via soft confusion matrix                         │
│  • Disabled early (epochs 1-10) for stability                       │
│                                                                      │
└─────────────────────────────────────────────────────────────────────┘

┌─────────────────────────────────────────────────────────────────────┐
│                L_cons: CONSISTENCY REGULARIZATION                    │
├────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  Enforce agreement between ordinal and regression predictions        │
│                                                                      │
│  L_cons = MSE(severity_ordinal, severity_regression)                │
│                                                                      │
│  Where:                                                              │
│    severity_ordinal = Σ_{k=0}^4 k · p_k  (expected value from CORN)       │
│    severity_regression = direct output from regression head         │
│                                                                      │
│  Properties:                                                         │
│  • Prevents heads from learning contradictory patterns               │
│  • Improves calibration of predictions                               │
│  • Weight λ_cons starts low (0.01) for stability                    │
│                                                                      │
└─────────────────────────────────────────────────────────────────────┘

═══════════════════════════════════════════════════════════════════════
                        📈 TRAINING CONFIGURATION (train.py)
═══════════════════════════════════════════════════════════════════════

┌─────────────────────────────────────────────────────────────────────┐
│                     PROGRESSIVE CURRICULUM LEARNING                  │
├────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  ACTUAL IMPLEMENTATION (25 Epochs Total Per Fold):                  │
│                                                                      │
│  ┌───────────────────────────────────────────────────────┐          │
│  │ STAGE 1: Coarse Learning (Epochs 1-6)                 │          │
│  │ ──────────────────────────────────────────────────     │         │
│  │ Resolution: 512×512 pixels                             │         │
```

```
│  │ Batch Size: 16                                          │          │
│  │ Grad Accumulation: 2                                    │          │
│  │ Effective Batch: 32                                     │          │
│  │ Iterations: ~2,415 per epoch                            │          │
│  │ Time: ~6 hours total (1 hour per epoch)                 │          │
│  │                                                         │          │
│  │ Purpose: Learn global fundus patterns                   │          │
│  │ • Optic disc location                                   │          │
│  │ • Major vessel structure                                │          │
│  │ • Overall image quality                                 │          │
│  └─────────────────────────────────────────────────────────┘          │
│                                                                       │
│  ┌─────────────────────────────────────────────────────────┐          │
│  │ STAGE 2: Medium-Resolution Refinement (Epochs 7-16)     │          │
│  │ ─────────────────────────────────────────────────────   │          │
│  │ Resolution: 768×768 pixels                              │          │
│  │ Batch Size: 12                                          │          │
│  │ Grad Accumulation: 2                                    │          │
│  │ Effective Batch: 24                                     │          │
│  │ Iterations: ~2,575 per epoch                            │          │
│  │ Time: ~11 hours total (1.1 hours per epoch)             │          │
│  │                                                         │          │
│  │ Purpose: Detect medium-sized lesions                    │          │
│  │ • Hemorrhages (50-200 pixels)                           │          │
│  │ • Exudates (hard/soft)                                  │          │
│  │ • Cotton wool spots                                     │          │
│  │ • Vessel caliber changes                                │          │
│  └─────────────────────────────────────────────────────────┘          │
│                                                                       │
│  ┌─────────────────────────────────────────────────────────┐          │
│  │ STAGE 3: High-Resolution Fine-Tuning (Epochs 17-25)     │          │
│  │ ─────────────────────────────────────────────────────   │          │
│  │ Resolution: 1024×1024 pixels                            │          │
│  │ Batch Size: 8                                           │          │
│  │ Grad Accumulation: 3                                    │          │
│  │ Effective Batch: 24                                     │          │
│  │ Iterations: ~2,415 per epoch                            │          │
│  │ Time: ~18 hours total (2.0 hours per epoch)             │          │
│  │                                                         │          │
│  │ Purpose: Detect microaneurysms and fine details         │          │
│  │ • Microaneurysms (10-30 pixels at 1024px)               │          │
│  │ • Small hemorrhages                                     │          │
│  │ • Subtle neovascularization                             │          │
│  │ • IRMA (intraretinal microvascular abnormalities)       │          │
│  └─────────────────────────────────────────────────────────┘          │
│                                                                       │
│  Total Training Time per Fold: ~35 hours                              │
│  Total for 5 Folds: ~175 hours (7.3 days)                             │
```

```
│                                                                              │
└──────────────────────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────────────────────┐
│                    DIFFERENTIAL LEARNING RATES                               │
├──────────────────────────────────────────────────────────────────────────────┤
│                                                                              │
│  Rationale: Different network components need different adaptation rates    │
│                                                                              │
│  CONVNEXTV2-LARGE (Folds 1, 3, 5):                                           │
│  ┌──────────────────────────────────────────────────────────────┐            │
│  │              │  512px     │  768px     │  1024px    │          │           │
│  │  Component   │  (Ep 1-6)  │  (Ep 7-16)│  (Ep17-25)│  Rationale│            │
│  ├──────────────┼────────────┼────────────┼────────────┼──────────┤           │
│  │ Backbone     │  8.0e-6    │  8.0e-6    │  1.0e-5    │ Pre-trained│          │
│  │              │            │            │            │ needs slow │          │
│  │              │            │            │            │ adaptation │          │
│  ├──────────────┼────────────┼────────────┼────────────┼──────────┤           │
│  │ FRG          │  2.5e-5    │  2.5e-5    │  3.0e-5    │ Task-     │           │
│  │              │            │            │            │ specific, │           │
│  │ CBAM         │  2.5e-5    │  2.5e-5    │  3.0e-5    │ faster    │           │
│  │              │            │            │            │ learning  │           │
│  │ Heads        │  2.5e-5    │  2.5e-5    │  3.0e-5    │           │           │
│  └──────────────┴────────────┴────────────┴────────────┴──────────┘           │
│                                                                              │
│  EFFICIENTNETV2-L (Folds 2, 4):                                              │
│  ┌──────────────────────────────────────────────────────────────┐            │
│  │              │  512px     │  768px     │  1024px    │          │           │
│  │  Component   │  (Ep 1-6)  │  (Ep 7-16)│  (Ep17-25)│  Rationale│            │
│  ├──────────────┼────────────┼────────────┼────────────┼──────────┤           │
│  │ Backbone     │  5.0e-6    │  5.0e-6    │  6.0e-6    │ Larger    │           │
│  │              │            │            │            │ pre-train │           │
│  │              │            │            │            │ (IN-21K)  │           │
│  ├──────────────┼────────────┼────────────┼────────────┼──────────┤           │
│  │ FRG          │  2.0e-5    │  2.0e-5    │  2.5e-5    │ Task-     │           │
│  │              │            │            │            │ specific  │           │
│  │ CBAM         │  2.0e-5    │  2.0e-5    │  2.5e-5    │           │           │
│  │ Heads        │  2.0e-5    │  2.0e-5    │  2.5e-5    │           │           │
│  └──────────────┴────────────┴────────────┴────────────┴──────────┘           │
│                                                                              │
│  LR Schedule: CosineAnnealingWarmRestarts                                    │
│  • T_0 = 10 epochs (restart period)                                          │
│  • T_mult = 2 (double period each restart)                                   │
│  • η_min = base_lr × 0.1 (minimum LR)                                        │
│                                                                              │
└──────────────────────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────────────────────┐
```

```
│                         OPTIMIZATION & REGULARIZATION                          │
├────────────────────────────────────────────────────────────────────────────────┤
│                                                                                │
│  Optimizer: AdamW                                                              │
│  ├─ β₁ = 0.9 (momentum)                                                        │
│  ├─ β₂ = 0.999 (RMSProp-like)                                                  │
│  ├─ weight_decay = 1e-4 (L2 regularization)                                    │
│  └─ ε = 1e-8 (numerical stability)                                             │
│                                                                                │
│  Mixed Precision Training (AMP):                                               │
│  ├─ Forward/backward in float16                                                │
│  ├─ Weights stored in float32                                                  │
│  ├─ Dynamic loss scaling                                                       │
│  └─ 2x speedup, 40% memory reduction                                           │
│                                                                                │
│  Gradient Clipping:                                                            │
│  ├─ Max norm = 1.0                                                             │
│  └─ Applied after gradient accumulation                                        │
│                                                                                │
│  Exponential Moving Average (EMA):                                             │
│  ├─ Decay = 0.999 (updated from 0.9999 for faster adaptation)                  │
│  ├─ Activated from epoch 3                                                     │
│  ├─ Separate shadow weights maintained                                         │
│  └─ Used for validation and final inference                                    │
│                                                                                │
│  Stochastic Depth (in backbone):                                               │
│  ├─ drop_path_rate = 0.3                                                       │
│  └─ Randomly drops residual connections during training                        │
│                                                                                │
│  Dropout (in heads):                                                           │
│  ├─ CORN head: 0.3                                                             │
│  ├─ Binary head: 0.3                                                           │
│  └─ Regression head: 0.2                                                       │
│                                                                                │
└────────────────────────────────────────────────────────────────────────────────┘

┌────────────────────────────────────────────────────────────────────────────────┐
│                           SAFETY & STABILITY FEATURES                          │
├────────────────────────────────────────────────────────────────────────────────┤
│                                                                                │
│  1. EMA Contamination Guard:                                                   │
│     • Detects mode collapse (QWK < 0.05)                                       │
│     • Reverts to best_ema checkpoint                                           │
│     • Cleans training log                                                      │
│     • Prevents corrupted weight propagation                                    │
│                                                                                │
│  2. Auto-Resume:                                                               │
│     • Saves latest checkpoint every epoch                                      │
```

```
│     • Detects incomplete training                                  │
│     • Resumes from last successful epoch                           │
│     • Preserves optimizer state and scheduler                      │
│                                                                     │
│  3. NaN Detection:                                                  │
│     • Checks for NaN in loss after each iteration                  │
│     • Logs occurrence and skips update                             │
│     • Prevents gradient explosion propagation                      │
│                                                                     │
│  4. Gradient Checkpointing:                                         │
│     • Trades compute for memory                                    │
│     • Enables larger batch sizes                                   │
│     • Recomputes activations during backward                       │
│                                                                     │
│  5. Early Stopping (disabled in final config):                     │
│     • patience = 10 epochs                                         │
│     • Monitors validation QWK                                      │
│     • Prevents overfitting in unstable training                    │
│                                                                     │
└────────────────────────────────────────────────────────────────────┘

═════════════════════════════════════════════════════════════════════
                    🔬 INFERENCE & POST-PROCESSING
═════════════════════════════════════════════════════════════════════

┌────────────────────────────────────────────────────────────────────┐
│                      TEST-TIME AUGMENTATION (TTA)                   │
├────────────────────────────────────────────────────────────────────┤
│                                                                     │
│  8 Augmented Views per Image:                                      │
│  1. Original                                                        │
│  2. Horizontal Flip                                                 │
│  3. Vertical Flip                                                   │
│  4. Rotate 90°                                                      │
│  5. Rotate 180°                                                     │
│  6. Rotate 270°                                                     │
│  7. H-Flip + V-Flip                                                │
│  8. H-Flip + Rotate 90°                                            │
│                                                                     │
│  Aggregation:                                                       │
│  • Average predictions across all 8 views                          │
│  • Reduces variance by ~30%                                        │
│  • Typical gain: +0.01 to +0.02 QWK                               │
│                                                                     │
└────────────────────────────────────────────────────────────────────┘

┌────────────────────────────────────────────────────────────────────┐
│                       THRESHOLD OPTIMIZATION                        │
```

```
├──────────────────────────────────────────────────────────────────────────────┤
│                                                                              │
│  OptimizedRounder (optimizer.py):                                            │
│                                                                              │
│  Problem: Default thresholds [0.5, 1.5, 2.5, 3.5] are suboptimal            │
│                                                                              │
│  Solution: Find thresholds that maximize validation QWK                      │
│                                                                              │
│  Algorithm:                                                                  │
│  1. Collect all validation predictions (continuous severity)                 │
│  2. Define search space: t₁ ∈ [0.3, 0.7]                                    │
│                         t₂ ∈ [1.3, 1.7]                                     │
│                         t₃ ∈ [2.3, 2.7]                                     │
│                         t₄ ∈ [3.3, 3.7]                                     │
│  3. Use scipy.optimize.minimize to find optimal thresholds                   │
│  4. Objective: Maximize QWK(predictions, targets)                            │
│                                                                              │
│  Typical Optimized Thresholds:                                               │
│  • t₁ ≈ 0.45 (Grade 0→1 boundary)                                           │
│  • t₂ ≈ 1.52 (Grade 1→2 boundary)                                           │
│  • t₃ ≈ 2.38 (Grade 2→3 boundary)                                           │
│  • t₄ ≈ 3.41 (Grade 3→4 boundary)                                           │
│                                                                              │
│  Typical Gain: +0.02 to +0.03 QWK                                           │
│                                                                              │
└──────────────────────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────────────────────┐
│                            ENSEMBLE PREDICTION                               │
├──────────────────────────────────────────────────────────────────────────────┤
│                                                                              │
│  5-Fold Weighted Ensemble (ensemble.py):                                     │
│                                                                              │
│  Weight Calculation:                                                         │
│    w_i = QWK_i² / Σ(QWK_j²)    for folds i ∈ {1,2,3,4,5}                  │
│                                                                              │
│  Prediction Aggregation:                                                     │
│    P_ensemble = Σ(w_i × P_i)                                                 │
│                                                                              │
│  Where P_i are the 5-class probability vectors from fold i                   │
│                                                                              │
│  Example Weights (if all folds achieve QWK ~0.82):                           │
│    Fold 1 (QWK 0.82): w₁ = 0.82²/Σ = 0.200                                 │
│    Fold 2 (QWK 0.82): w₂ = 0.82²/Σ = 0.200                                 │
│    Fold 3 (QWK 0.82): w₃ = 0.82²/Σ = 0.200                                 │
│    Fold 4 (QWK 0.82): w₄ = 0.82²/Σ = 0.200                                 │
│    Fold 5 (QWK 0.82): w₅ = 0.82²/Σ = 0.200                                 │
│                                                                              │
```

```
│  Final Class Prediction:                                              │
│    class = argmax(P_ensemble)                                         │
│                                                                       │
│  Typical Gain from Ensemble: +0.03 to +0.05 QWK vs best single fold   │
│                                                                       │
└───────────────────────────────────────────────────────────────────────┘

════════════════════════════════════════════════════════════════════════
                         📊 PERFORMANCE METRICS & RESULTS
════════════════════════════════════════════════════════════════════════

┌───────────────────────────────────────────────────────────────────────┐
│                         PRIMARY METRIC: QWK                           │
├───────────────────────────────────────────────────────────────────────┤
│                                                                       │
│  Quadratic Weighted Kappa (Cohen's Kappa)                             │
│                                                                       │
│  Interpretation Scale:                                                │
│  • 1.00      : Perfect agreement                                      │
│  • 0.93-0.99 : Exceptional (competition winning)                      │
│  • 0.90-0.92 : Excellent (target range)                               │
│  • 0.85-0.89 : Very Good                                              │
│  • 0.80-0.84 : Good                                                   │
│  • 0.75-0.79 : Moderate                                               │
│  • <0.75     : Needs improvement                                      │
│                                                                       │
└───────────────────────────────────────────────────────────────────────┘

┌───────────────────────────────────────────────────────────────────────┐
│                       ACTUAL TRAINING RESULTS                         │
├───────────────────────────────────────────────────────────────────────┤
│                                                                       │
│  FOLD 1 (ConvNeXt, 25 epochs):                                        │
│  ├─ Epoch 6 (512px):  QWK 0.00  (collapsed)                           │
│  ├─ Epoch 16 (768px): QWK 0.82                                        │
│  ├─ Epoch 25 (1024px): QWK 0.8213                                     │
│  └─ Issues: Early collapse (Ep 3-6), recovered at 768px               │
│                                                                       │
│  FOLD 2 (EfficientNet, 20 epochs, NO 1024px):                         │
│  ├─ Epoch 6 (512px):  QWK 0.76                                        │
│  ├─ Epoch 12 (768px): QWK 0.84                                        │
│  ├─ Epoch 20 (768px): QWK 0.8156                                      │
│  └─ Issues: Epoch 3 dip (0.49), recovered at Ep 4                     │
│                                                                       │
│  FOLD 3 (ConvNeXt, stopped at Epoch 4):                               │
│  ├─ Epoch 1: QWK 0.71                                                 │
│  ├─ Epoch 2: QWK 0.77                                                 │
│  ├─ Epoch 3: QWK 0.68 (dip)                                           │
```

```
│  ├─ Epoch 4: QWK 0.75 (recovering)                                    │
│  └─ Training stopped for reconfiguration                               │
│                                                                         │
│  KEY OBSERVATIONS:                                                      │
│  • All folds plateau around 0.82 QWK                                   │
│  • Weighted sampler caused instability (Epoch 3 dips)                  │
│  • 1024px stage with BS=4 didn't improve beyond 768px                 │
│  • Need: BS=8 at 1024px + longer training (30 epochs)                 │
│                                                                         │
└───────────────────────────────────────────────────────────────────────┘


┌───────────────────────────────────────────────────────────────────────┐
│                        EXPECTED FINAL PERFORMANCE                       │
├───────────────────────────────────────────────────────────────────────┤
│                                                                         │
│  With Optimized Configuration (BS=8 at 1024px, 25-30 epochs):         │
│                                                                         │
│  Per-Fold Targets:                                                      │
│  ├─ Single Fold: 0.89-0.91 QWK                                        │
│  ├─ Sensitivity: 0.70-0.75                                            │
│  └─ Specificity: 0.96-0.98                                            │
│                                                                         │
│  5-Fold Ensemble:                                                       │
│  ├─ Raw Ensemble: 0.87-0.89 QWK                                       │
│  ├─ + TTA (8x): 0.88-0.90 QWK                                         │
│  └─ + Optimized Thresholds: 0.89-0.91 QWK ✅                          │
│                                                                         │
│  FINAL TARGET: 0.90 QWK (ACHIEVABLE)                                   │
│                                                                         │
└───────────────────────────────────────────────────────────────────────┘


═══════════════════════════════════════════════════════════════════════════
                          💾 MODEL ARTIFACTS & OUTPUTS
═══════════════════════════════════════════════════════════════════════════

results/
├── fold_1/
│   ├── convnextv2_large_best_ema.pth          (Best model weights)
│   ├── convnextv2_large_latest.pth            (Auto-resume checkpoint)
│   ├── training_log.csv                       (Complete training history)
│   └── optimized_thresholds.json              (Post-training threshold optimization)
├── fold_2/
│   ├── efficientnetv2_l_best_ema.pth
│   ├── efficientnetv2_l_latest.pth
│   ├── training_log.csv
│   └── optimized_thresholds.json
├── fold_3/
├── fold_4/
```

```
├── fold_5/
└── ensemble_predictions.csv                    (Final predictions)

════════════════════════════════════════════════════════════════════════════════
                              🔧 SYSTEM REQUIREMENTS
════════════════════════════════════════════════════════════════════════════════

Hardware:
├── GPU: NVIDIA RTX A4000 (16GB VRAM)
├── CPU: Multi-core (10+ workers for data loading)
├── RAM: 32GB+ recommended
└── Storage: 250GB+ for preprocessed images and checkpoints

Software:
├── Python: 3.10
├── PyTorch: 2.0+
├── CUDA: 11.8+
├── timm: 1.0.25 (model library)
├── albumentations: Latest (data augmentation)
└── Additional: numpy, pandas, opencv, scikit-learn

Training Time:
├── Per Fold: ~35 hours (25 epochs)
├── 5 Folds: ~175 hours (7.3 days)
└── With optimization: ~62 hours (2.6 days) for 2 folds

════════════════════════════════════════════════════════════════════════════════
```